



Algoritmi de sortare

Bubble · Selection · Insertion · STL · Counting Sort

Andrei Mihai Ciobanu

Algolymp Summer School

De ce sortăm?

Bubble Sort

Selection Sort

Insertion Sort

STL sort

Counting Sort

Comparație

Recapitulare

Resurse

Probleme de antrenament

De ce sortăm?

De ce sortăm?

Sortarea este una dintre cele mai frecvente operații în programare.

- Căutarea binară necesită un șir sortat
- Multe probleme devin triviale după sortare
(ex: al k -lea cel mai mare element, perechi cu suma dată)
- Date reale: liste de note, clasamente, ierarhii

Problema sortării

Dat un vector $v[0..n-1]$, să se rearanjeze elementele astfel încât $v[0] \leq v[1] \leq \dots \leq v[n-1]$.

Bubble Sort

Bubble Sort — Idee

Idee: la fiecare pas, comparăm perechi de elemente adiacente și le schimbăm dacă sunt în ordinea greșită.

Exemplu: [5, 3, 8, 1, 4]

- Pasul 1: [3, 5, 8, 1, 4] → [3, 5, 1, 8, 4] → [3, 5, 1, 4, 8]
- Pasul 2: [3, 1, 5, 4, 8] → [3, 1, 4, 5, 8]
- Pasul 3: [1, 3, 4, 5, 8]
- Pasul 4: sortat ✓

După pasul i , ultimele i elemente sunt la locul lor definitiv.

Bubble Sort — Implementare

```
1 void bubbleSort(vector<int>& v) {  
2     int n = v.size();  
3     for (int i = 0; i < n - 1; i++) {  
4         for (int j = 0; j < n - 1 - i; j++) {  
5             if (v[j] > v[j + 1]) {  
6                 swap(v[j], v[j + 1]);  
7             }  
8         }  
9     }  
10 }
```

- Bucla exterioară: $n - 1$ pași
- Bucla interioară: la pasul i facem $n - 1 - i$ comparații
- Total comparații: $(n - 1) + (n - 2) + \dots + 1 = \frac{n(n-1)}{2}$

Bubble Sort — Optimizare și Complexitate

Optimizare: repetăm etapa de corectare a inversiunilor până când nu mai are loc nicio inversare.

```
1 void bubbleSort(vector<int>& v) {  
2     int n = v.size();  
3     bool ok = true; // exista inversiuni  
4     while (ok) {  
5         ok = false; // presupunem ca nu mai sunt  
6         for (int j = 0; j < n - 1; j++) {  
7             if (v[j] > v[j + 1]) {  
8                 swap(v[j], v[j + 1]);  
9                 ok = true; // am gasit o inversiune  
10            }  
11        }  
12    }  
13 }
```

- **Best case** (deja sortat): $O(n)$ — un singur pas, nicio inversare
- **Worst case** (sortat invers): $O(n^2)$
- **Average case**: $O(n^2)$

Selection Sort

Selection Sort — Idee

Idee: la fiecare pas, găsim minimul din partea nesortată și îl aducem la începutul ei.

Exemplu: [5, 3, 8, 1, 4]

- Pasul 0: min = 1 → swap: [1, 3, 8, 5, 4]
- Pasul 1: min = 3, deja la loc: [1, 3, 8, 5, 4]
- Pasul 2: min = 4 → swap: [1, 3, 4, 5, 8]
- Pasul 3: min = 5, deja la loc: [1, 3, 4, 5, 8]
- Pasul 4: sortat ✓

După pasul i , primele $i + 1$ elemente sunt la locul lor definitiv.

Selection Sort — Implementare

```
1 void selectionSort(vector<int>& v) {  
2     int n = v.size();  
3     for (int i = 0; i < n - 1; i++) {  
4         for (int j = i + 1; j < n; j++) {  
5             if (v[j] < v[i]) {  
6                 swap(v[i], v[j]);  
7             }  
8         }  
9     }  
10 }
```

- **Best case:** $O(n^2)$ — mereu facem $\frac{n(n-1)}{2}$ comparații
- **Worst case:** $O(n^2)$
- **Average case:** $O(n^2)$

Insertion Sort

Insertion Sort — Idee

Idee: construim un prefix sortat, inserând pe rând fiecare element la locul potrivit.

Exemplu: $[5, 3, 8, 1, 4]$ ($|$ = limita prefix sortat)

- $[5 | 3, 8, 1, 4]$: inserăm 3 $\rightarrow [3, 5 | 8, 1, 4]$
- $[3, 5 | 8, 1, 4]$: inserăm 8 $\rightarrow [3, 5, 8 | 1, 4]$
- $[3, 5, 8 | 1, 4]$: inserăm 1 $\rightarrow [1, 3, 5, 8 | 4]$
- $[1, 3, 5, 8 | 4]$: inserăm 4 $\rightarrow [1, 3, 4, 5, 8]$ ✓

Elementul inserat este mutat la stânga cât timp e mai mic decât vecinul său.

Insertion Sort — Implementare

```
1 void insertionSort(vector<int>& v) {  
2     int n = v.size();  
3     for (int i = 1; i < n; i++) {  
4         int j = i;  
5         while (j >= 0 && v[j - 1] > v[j]) {  
6             swap(v[j - 1], v[j]);  
7             j--;  
8         }  
9     }  
10 }
```

- **Best case** (deja sortat): $O(n)$ — nicio deplasare
- **Worst case** (sortat invers): $O(n^2)$
- **Average case**: $O(n^2)$

STL sort

STL sort — Utilizare de bază

C++ oferă `std::sort` în header-ul `<algorithm>`. Complexitate acestuia este $O(n \log n)$.

```
1 #include <algorithm>
2 #include <vector>
3 using namespace std;
4
5 vector<int> v = {5, 3, 8, 1, 4};
6
7 // Sortare crescatoare (implicit)
8 sort(v.begin(), v.end());
9 // v = {1, 3, 4, 5, 8}
10
11 // Sortare descrescatoare
12 sort(v.begin(), v.end(), greater<int>());
13 // v = {8, 5, 4, 3, 1}
14
15 // Sortare pe un subarray [l, r]
16 sort(v.begin() + l, v.begin() + r + 1);
```

▷ *vector, begin, end* — din STL (vezi slide-ul Resurse)

STL sort — Comparatori custom

Putem sorta după orice criteriu printr-o funcție de comparare.

```
1 // Sortare dupa valoare absoluta
2 bool cmpAbs(int a, int b) {
3     return abs(a) < abs(b);
4 }
5
6 sort(v.begin(), v.end(), cmpAbs);
7
8 vector<pair<int, int>> pv{ { 1, 2 }, { 1, 3 }, { 2, 1 } };
9
10 // Sortare perechi: crescator dupa primul element,
11 // in caz de egalitate descrescator dupa al doilea
12 bool cmpPair(pair<int,int> a, pair<int,int> b) {
13     if (a.first != b.first)
14         return a.first < b.first;
15     return a.second > b.second;
16 }
17
18 sort(pv.begin(), pv.end(), cmpPair);
```

▷ *pair, abs — din STL (vezi slide-ul Resurse)*

Counting Sort

Counting Sort — Idee

Idee: nu comparăm elementele — numărăm de câte ori apare fiecare valoare, apoi reconstruim șirul.

Condiție: valorile sunt întregi dintr-un interval $[0, k]$ cu k rezonabil de mic.

Exemplu: $v = [3, 1, 4, 1, 5, 3, 3]$, $k = 5$

Valoare	0	1	2	3	4	5
Count	0	2	0	3	1	1

Rezultat: $[1, 1, 3, 3, 3, 4, 5]$

Counting Sort — Implementare

```
1 void countingSort(vector<int>& v) {  
2     // Numaram aparitiile fiecarei valori  
3     int n = v.size();  
4     int maxVal = *max_element(v.begin(), v.end());  
5     vector<int> cnt(maxVal + 1, 0);  
6     for (int i = 0; i < n; i++) {  
7         cnt[v[i]]++;  
8     }  
9     // Reconstruim vectorul sortat  
10    int idx = 0;  
11    for (int val = 0; val <= maxVal; val++) {  
12        for (int i = 0; i < cnt[val]; i++) {  
13            v[idx++] = val;  
14        }  
15    }  
16 }
```

- Total: $O(n + k)$ — n pentru numărare, k pentru parcurgerea valorilor posibile

▷ `max_element` — din STL (vezi slide-ul Resurse)

Când funcționează

- Valori întregi nenegative
- k mic (ex: note 1–10, vârste 0–150, valori $\leq 10^6$)

Când NU funcționează

- Valori negative sau reale (trebuie transformate mai întâi)
- k foarte mare (ex: valori până la 10^9 — ar necesita 4 GB memorie)

Comparație

Comparație algoritmi de sortare

Algoritm	Timp	Spațiu	Condiții
Bubble Sort	$O(n^2)$	$O(1)$	orice tip comparabil
Selection Sort	$O(n^2)$	$O(1)$	orice tip comparabil
Insertion Sort	$O(n^2)$	$O(1)$	orice tip comparabil
STL sort	$O(n \log n)$	$O(\log n)$	orice tip comparabil
Counting Sort	$O(n + k)$	$O(k)$	întregi în $[0, k]$

- În practică: folosește **STL sort** dacă nu ai restricții speciale
- Counting Sort bate STL sort când $k = O(n)$
- Bubble Sort — util pedagogic, evitat în concursuri

Recapitulare

Recapitulare

Operație	Cod C++
Sortare crescătoare	<code>sort(v.begin(), v.end())</code>
Sortare descrescătoare	<code>sort(v.begin(), v.end(), greater<int>())</code>
Sortare subarray	<code>sort(v.begin() + l, v.begin() + r + 1)</code>
Comparator custom	<code>sort(v.begin(), v.end(), cmp)</code>

- **Bubble Sort / Selection Sort / Insertion Sort:** $O(n^2)$
- **STL sort:** $O(n \log n)$, soluția implicită
- **Counting Sort:** $O(n + k)$, doar pentru întregi mici

Resurse

Sortare

- [pbinfo.ro](#) — Sortarea tablourilor (RO)
- [pbinfo.ro](#) — Sortare prin selecție (RO)
- [cp-algorithms.com](#) — Bubble Sort (EN)
- [cppreference.com](#) — `std::sort` (EN)

STL

- [pbinfo.ro](#) — STL în română

Bonus — algoritmi mai eficienți

- [wikipedia.org](#) — Quicksort, $O(n \log n)$ mediu (EN)
- [wikipedia.org](#) — Merge Sort, $O(n \log n)$ garantat (EN)

Probleme de antrenament

Ușoară

- **Even Sort** (AlgoCode)

Medii

- **K Sort** (AlgoCode)
- **Sort Primes** (AlgoCode)

Grea

- **Array Task** (AlgoCode)